

Model Validation Using Invariant Signatures and Logic-Based Inference for Automated Building Code Compliance Checking

Jin Wu, S.M.ASCE¹; and Jiansong Zhang, Ph.D., A.M.ASCE²

Abstract: Fully automated building code compliance checking (ACC) requires accurate information extraction from both building information models (BIMs) and building code chapters, and equally (if not more) importantly, a precise matching between the two. Although research on information extraction has been extensively conducted for ACC, there is a lack of investigation of automated and practical information mapping between the extracted information, from BIMs to building code requirements. To address this gap, the authors proposed a new method for BIM model validation to validate an input Industry Foundation Classes (IFC) model with regard to building code concepts. This validation method was supported by creating invariant signatures of architecture, engineering, and construction (AEC) objects that capture the geometric nature of the objects. Target concepts from building codes are classified into four categories: (1) explicit concepts, (2) inferable concepts, (3) user-assisted concepts, and (4) system defaults. Identification algorithms are developed for all four categories based on the invariant signatures of AEC objects. An experiment was conducted to test the proposed method on validating five real commercial project models with selected concepts from a current building code. Compared with a manually developed gold standard, 99.8% precision and 99.6% recall were achieved. This demonstrates that the proposed method is promising in supporting information matching between BIMs and building code concepts for ACC purpose. **DOI:** [10.1061/\(ASCE\)CP.1943-5487.0001002](https://doi.org/10.1061/(ASCE)CP.1943-5487.0001002). © 2022 American Society of Civil Engineers.

Introduction

Building information modeling (BIM) is very promising in the architecture, engineering, and construction (AEC) domain (Azhar 2011). It helps information conversion and communication in a more effective way. Compared with the traditional method of manual building code compliance checking, automated compliance checking (ACC) that utilizes BIM technologies is expected to be more efficient in time and cost, and tends to generate fewer errors (Nguyen and Kim 2011). However, current BIM technology, such as industry foundation classes (IFC), only supports limited information coverage related to building codes (Zhang and El-Gohary 2016a). The seamless linkage between the IFC standard and building codes is missing. More specifically, the concepts in building codes mostly do not have a direct mapping to the entities defined in IFC standards and thus cannot be directly matched with IFC models in checking the model compliance.

To allow the full automation of the model compliance checking process, several efforts have been made. For example, Tan et al. (2010) proposed a framework of automated code compliance checking for building envelope design. Yang and Xu (2004) proposed a software implementation of knowledge modeling for building code compliance checking. Zhang and El-Gohary (2016b) proposed a

method to extend the IFC schema with extracted regulatory concepts. Xu and Cai (2020) proposed to convert BIM data to an ontology for matching/checking with spatial constraints in utilities regulatory documents. In spite of the advancement made by these existing efforts, however, there is still the lack of a systematic investigation of the feasibility of automation in matching BIMs with building code concepts at the instance level and from the practical point of view.

To address this gap, the authors propose a new method to conduct BIM model validation by automatically identifying building code concepts from the models using invariant signatures and logic-based inference. In this new method, the explicit information that could be directly extracted from BIMs is further used to infer more types of concepts (i.e., information extension) with the help of minimal information input from the users, for arriving at a set of computable building design data that can be directly matched to building code concepts. To represent the extended information, logic facts were generated for their storage. The proposed method does not need to modify or extend the existing schema of the IFC standard in use. Instead, it creates a new information set using invariant signatures that can (1) validate all the explicit information from the IFC-based BIMs, (2) check for missing information, (3) infer target information, and (4) solicit uninferable information from users for mapping to corresponding concepts from building codes. For the missing information, only minimal input from users is required through the help of inference making of intermediate data that brings BIM data closer to target concepts. The resulting logic facts from the proposed method also ease the integration into building code compliance checking systems.

Background

Domain Knowledge Representations of Building Codes

To enable automated building code compliance checking, one of the main steps is to convert the building codes written in

¹Research Assistant, Automation and Intelligent Construction (AutoIC) Lab, School of Construction Management Technology, Purdue Univ., West Lafayette, IN 47907. Email: wu1275@purdue.edu

²Founder and Director, Automation and Intelligent Construction (AutoIC) Lab, School of Construction Management Technology, Purdue Univ., West Lafayette, IN 47907 (corresponding author). ORCID: <https://orcid.org/0000-0001-5225-5943>. Email: zhan3062@purdue.edu

Note. This manuscript was submitted on April 6, 2021; approved on September 23, 2021; published online on February 7, 2022. Discussion period open until July 7, 2022; separate discussions must be submitted for individual papers. This paper is part of the *Journal of Computing in Civil Engineering*, © ASCE, ISSN 0887-3801.

natural/human language into computing language. In discussions of natural language, syntax refers to the word sequences, semantics refers to the sense and meaning, and pragmatics refers to different interpretations in different contexts (Nawari 2018). To enable machines to process such syntax, semantics, and pragmatics, natural language processing (NLP) (Nadkarni et al. 2011) methods were developed and used. The NLP approach has been shown to be promising in converting building code rules and infrastructure regulations written in plain text into computable representations (Zhang and El-Gohary 2016b; Xu and Cai 2020).

Regarding computable representations of building codes, much research has been done in representing the building code requirements in various computable formats. For example, Khemlani (2011) proposed to use predicate logic to represent building codes in FORNAX (i.e., a C++ library for IFC data editing). Ding et al. (2006) proposed to use rule-based language to represent accessibility requirements in building codes. Martins and Monteiro (2013) proposed to use XML-based language to represent building codes of water systems. Tan et al. (2010) proposed to use XML-based language to represent building codes for checking building envelope using the Jboss rule engine.

In addition, Jeong and Lee (2009) proposed to use direct hard coding to check building fire resistance requirements automatically. Nawari (2012) proposed to use object-oriented representation for encoding a knowledge domain. Zhang and El-Gohary (2016c) proposed to use first-order logic (FOL) for encoding building codes. FOL consists of objects, relations, and functions. In recent research, domain-specific ontology was also used in constructing a set of semantic and syntactic structures for building codes (Zhang and El-Gohary 2016c). Independent from such computable representations, however, is the need of matching BIMs to related building code concepts.

Automated Code Compliance Checking

The first successful effort for code compliance checking can be traced back to the 1960s when Fenves (1966) designed an if-then system to represent AISC standard specifications. Later, many research efforts followed and made advancement in automated code compliance checking (Lopez and Wright 1985; Lopez et al. 1989; Garrett and Fenves 1987). In recent studies (Holzer 2015; Sionov et al. 2015; Volk et al. 2014; Zou et al. 2017; Sacks 2018), researchers showed great interests in using BIM to support code compliance checking, which is expected to be interoperable among platforms for designers, contractors, clients, vendors, and others.

An important problem each and every BIM-based ACC system must address is how to match building design information from BIMs to building code concepts that are essential building blocks of regulatory rules in building codes. This was reflected in the four-stage rule checking framework that Eastman et al. (2009) summarized as including

“(1) rule interpretation and logical structuring of rules for their application; (2) building model preparation, where the necessary information required for checking is prepared; (3) the rule execution phase, which carries out the checking; and (4) the reporting of the checking results.”

The first two stages are intended to prepare input (i.e., building design and building codes, respectively) for the third phase, rule execution. In order for the building code rules to execute over the building design input, there lies the matching between the two inputs.

In tackling this problem, recent ACC efforts mainly focused at the rule level (i.e., regulatory requirements from building codes).

For example, Kasim et al. (2013) proposed a reusable solution for sustainability compliance checking using the requirement-applicabilities-selection-exception (RASE) methodology developed by Hjelseth and Nisbet (2011). Their method can extract sustainability requirements and convert them into compiled rules for use with a rule engine. It has the potential to support dynamic checking during the building design stage. Solihin and Eastman (2015) proposed to classify the rules in building codes into seven categories, including (1) rules for checking the well-formedness of a building model, (2) rules for building regulatory code checking, (3) rules for specific client requirements; (4) rules for constructability/other contractor requirements, (5) rules for safety/other rules with corrective actions, (6) rules for warranty approvals, and (7) rules for checking BIM data completeness.

Zhang and El-Gohary (2017) proposed a semantic NLP and logic reasoning-based method to support fully automated code compliance checking. Their prototype achieved 98.7% recall and 87.6% precision in noncompliance detection in Chapter 19 of the ICC (2009), which was based on representing each regulatory requirement as a logic rule. Haubler et al. (2021) proposed a rule-based method to implement the code compliance checking of railways, with 12 categories of rules (e.g., component definitions and directional definitions). Their approach was shown to be able to automatically examine 37%–75% of the 943 rules.

BIM has a great potential in supporting automated code compliance checking, especially with the support of IFC standard, which is open and platform-neutral. However, despite the existing efforts and progresses achieved, practical obstacles remain in information extraction and matching from both BIMs and specific building codes at the regulatory concept level. For example, BIMs are not expected to contain explicit depictions of egress, which is an important concept in building codes.

Information Exchange in Building Information Models

Information exchange has been frequently studied (Sarawagi 2008; Yang et al. 2019), especially using the ontology-based approach in recent research (Fernández et al. 2011; Yehia et al. 2019; Paliouras et al. 2011; Wimalasuriya and Dou 2010). For information exchange in BIMs in a design science research study carried out by Ding et al. (2017), a real-time quality checking system was accomplished using an industrial foundation classes-based inspection process model (IFC-IPM). Their system demonstrated the efficiency in quality information exchange and could be used in inspection activities.

Kim et al. (2016) developed a new approach to extract and process material information in BIMs to explore energy-saving options early in the design phase. The major limitation of such an approach was the inaccuracies during simplifications in construction/material data. Their new system consisted of three parts, namely information extraction, material property matching, and file writing. The system improved the efficiency of energy modeling by eliminating the manual information input and increased the accuracy using the developed matching algorithms.

For information exchange of IFC models in automated code compliance checking, Zhang and El-Gohary (2019) proposed a machine learning-based approach to map building code concepts and relations to IFC elements and relations. Their method achieved 77% matching accuracy in IFC elements and 78% accuracy in IFC relations, which is representing the state of the art. For practical building code compliance checking automation, performance improvement will be needed. In addition, many concepts in building codes do not have a one-to-one mapping to IFC entities. The bridge

between building codes concepts and IFC entities still need to be built in additional ways.

Identification of Building Components

There is also no lack of research in automated identification of building components. For example, Quintana et al. (2018) proposed a method for door detection from three-dimensional (3D) colored point-cloud data. Their approach could detect open, semi-open, and closed doors from the laser scanned data of an indoor environment by integrating geometry, colors, and other characteristics into the detection analysis. As a result, they were able to detect doors with close to 100% precision and higher than 90% recall. Adán et al. (2018) proposed a method for detecting secondary building components [e.g., mechanical, electrical and plumbing (MEP) components] from laser scanners. They proposed a six-dimensional (6D) approach [XYZ + red, green, and blue (RGB)] to recognize small objects such as switches and signs by inferring the objects following a consensus procedure, to create an as-is semantically rich 3D model.

Puente et al. (2014) proposed a method for detecting road tunnel luminaires using mobile light detection and ranging (LiDAR) technology. Hamledari et al. (2017) proposed a method to detect components of under-construction indoor partitions using computer vision-based technologies. It was found that the methods for detecting and constructing as-is building components have been studied extensively with practical results.

However, reasoning about semantic building concepts (e.g., egress) from as-design models to produce reliable and practical results was underresearched, as discussed in the “Information Exchange in Building Information Models” section. The detection of building code concepts is also different from detecting objects, because the building code concepts may involve multiple objects, relationships between them, and their combinations. As a result, new methods need to be developed to help identify these building code concepts, which the existing methods did not cover.

Proposed Method for ACC-Oriented BIM Validation

The authors propose an iterative method to extend the information from BIMs to better support information mapping to building codes for use in ACC systems. Different from previous approaches that directly map the building code concepts to IFC entities, which still do not provide enough information for compliance checking, or approaches that extend the IFC schema, which requires deep

knowledge and understanding of IFC standard, the authors propose to use invariant signatures of AEC objects (Wu et al. 2021) as intermediate results to connect building code concepts and IFC entities, and extend the information from IFC models to infer more concepts based on the invariant signatures. This will enable better mapping of the information in IFC-based BIMs to building code concepts (Fig. 1).

The authors propose an iterative approach to develop an intermediate information set with extended information added to BIMs to provide information mapping between BIMs and building code concepts, which in turn better supports automated compliance checking compared with the state of the art (Fig. 2). In this proposed method, there are four main steps:

1. construct invariant signatures for AEC objects from IFC models,
2. identify and classify target concepts from logic rules that represent regulatory information from building codes,
3. develop algorithms for extending model information to match target concepts, and
4. generate logic facts to store the extended information.

The algorithms are developed following a data-driven approach so that the algorithms can be continuously developed to cover more concepts and model representations. New rules and algorithms can be added under the condition that they do not interfere with previous results, to ensure compatibility and robustness. With more training data, the system will be more robust. It is not our goal in this paper, however, to build the ultimate all-in-one set of algorithms that cover each and every possible concept in building codes. Our goal in this paper is to introduce the method and framework to enable that.

Step 1: Construct Invariant Signatures for AEC Objects from IFC Models

To allow the information matching in later steps, this step extracts information from the BIMs. During this processing of the BIMs data, invariant signatures (Wu et al. 2021) are used to convert the IFC models into value entries of a set of predefined features, a data format that is easier to process and use. Invariant signatures are “a set of intrinsic properties of the object that distinguish it from others and that do not change with data schema, software implementation, modeling decisions, and/or language/cultural contexts” (Wu et al. 2021). They capture the geometric nature of the IFC model and can fully represent the AEC objects in the IFC models. In this proposed method, invariant signatures are extracted from BIMs using the IFC2x3 standard, which is commonly used in

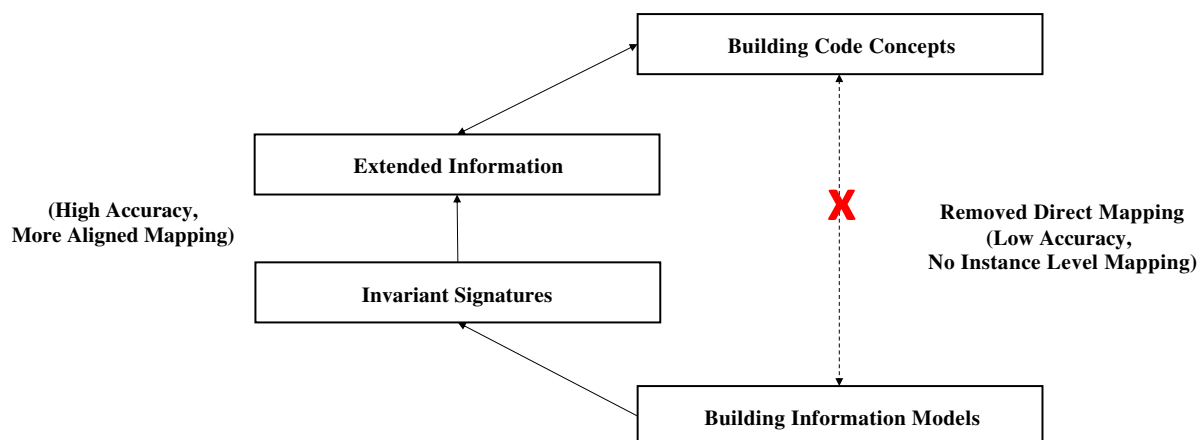


Fig. 1. Idea illustration of the proposed method.

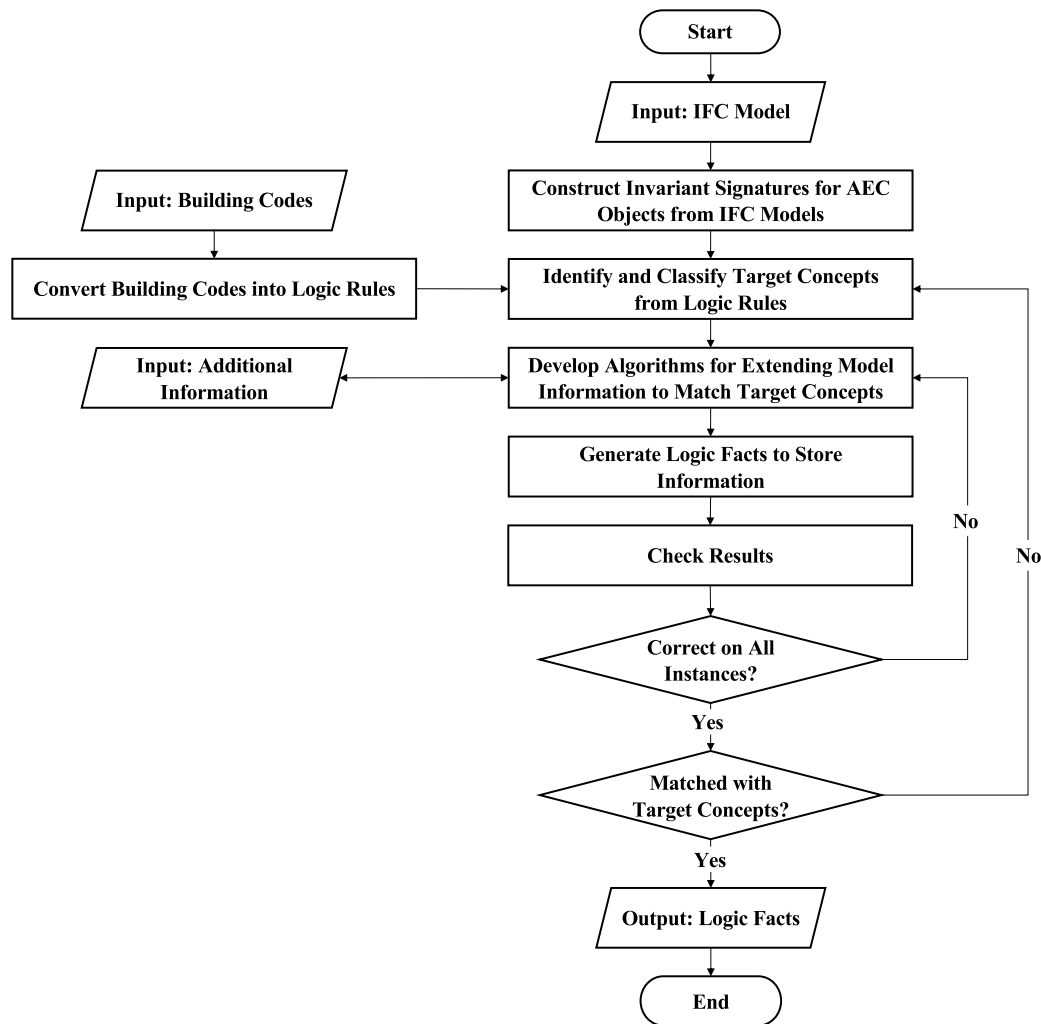


Fig. 2. Workflow of the proposed method.

commercial projects. Although invariant signatures are platform-neutral and independent from specific data schema, during their use, they still need to be extracted from practical models that follow certain data standards. For example, consistent invariant signatures must be generated from different representations of the same shape, e.g., boundary representations (Brep) and swept solid representations, two of the most commonly used 3D representations in IFC. In addition to these commonly used representations, a data-driven approach is used to develop algorithms for extracting information from the models that follow uncommon representations. In summary, the state-of-the-art invariant signatures extraction algorithms by Wu and Zhang (2019a, b) and Wu et al. (2021) are extended to allow full support for information extraction and information exchange for BIM model validation in this paper.

Experimental testing requires different BIMs as training data and testing data. The performance in terms of precision and recall is reported using element-level assessment, and the results are manually verified. During the training phase, the results shall achieve 100% in information extraction from the training models, and then the trained algorithms are tested on the testing models to assess their performance.

Step 2: Identify and Classify Target Concepts

The target concepts are extracted from logic rules that are in turn generated from building codes (regulations) using the state-of-the-art

regulatory information extraction and transformation algorithms (Zhang and El-Gohary 2015, 2016b). These algorithms can generate logic rules based on the building codes fully automatically. Because the state-of-the-art algorithms are not 100% accurate yet, the generated logic rules are further manually verified and modified as necessary to form gold standards. This is still much more efficient compared with generating the logic rules solely manually from scratch. In an experimental trial to generate the logic rules by comparing not using and using the machine-generated results, the average time for generating one logic rule was reduced from 321.2 to 84.2 s/rule (i.e., 73.8% time saving) by using the machine-generated results.

In the gold standard logic rules generated from the building codes, the logic clause in the form of *entity(Entity)* (where the predicate name has the same string with the argument but in lowercase) indicates a declaration of a concept instance. An observation of the building codes and those logic clauses showed that these predicates contain all the concepts in the corresponding sections of building codes, which are the target concepts to be mapped to. On the other hand, because almost all nouns are generated in this form of instance declaration, the generated logic rules contain multiple types of building code concepts. For example, building, door opening, Group H, and Section 10.4 are all concepts generated from the building codes, which are of completely different types of concepts. To allow seamless information mapping, it is critical to classify

these concepts, and then develop rule-based identification algorithms based on the characteristics of each type.

In the proposed method, the authors classify the concepts into four types, which are explicit concept, inferable concept, user-assisted concept, and system default, respectively. These target concepts are classified by developers, and end-users are not required to participate in this classification process.

Explicit concepts are the concepts that are directly generable from the BIMs, i.e., the concept has a one-to-one mapping to IFC entities and can be directly identified. For example, a wall concept has corresponding AEC objects in IFC entities: *IfcWall* and *IfcWallStandardCase*.

Nonexplicit concepts are more challenging in their detection compared with the explicit concepts because there is no direct one-to-one mapping between the concepts and IFC entities. However, some nonexplicit concepts can be inferred based on related information in the IFC model. Accordingly, there are two subcategories: inferable and user-assisted. If a consistent inference rule can be found, then that concept is considered inferable, i.e., the inferable concepts are the concepts that can be heuristically inferred from the explicit information in the model with consistency; if not, then that concept is considered to require user judgement and therefore is classified as user-assisted. For example, egress is considered an inferable concept whereas bathroom is classified as a user-assisted concept.

The identification of inferable concepts is performed by recognizing related IFC entities and then selecting those that satisfy additional constraints based on heuristic rules. The additional constraints leverage the geometry and relative locations of these IFC entities compared with other IFC entities in the same model. For each inferable concept, one heuristic rule-based algorithm is developed. The algorithms are developed by analyzing the corresponding human recognition process to summarize needed heuristics, and then digitalizing and formalizing these heuristics into rules. The rules are then verified on the training data to validate the correctness of the identification algorithms during the training process.

One of the most representative inferable concepts is egress. For identifying an egress, the main heuristic was to identify the exit doors of the building. Therefore, the constraints used in the egress identification algorithm were to find doors that locate at the boundary of the building that connect interior and exterior of the building. Based on the geometric and locational information, interior doors can be excluded from the egress. During this process, all the reasoning can be performed automatically without human judgment, with the developed heuristic rules. Although it is not difficult for a human to judge if a door is an egress, the algorithm for identifying these requirements still needs to be developed with exact steps to ensure the algorithm can function well to generate the expected results robustly.

In contrast, user-assisted concepts are the concepts that can be semi-inferable with additional user input and judgement. The concepts that rely solely on user input could be straightforwardly handled by taking user inputs and therefore are outside of the scope of this paper. Although logically, it is straightforward to demonstrate if a concept is inferable (i.e., by finding a way to infer it), it is close to impossible to prove if a concept is not inferable. The boundary between inferable and user-assisted concepts therefore depends on the current implementation and reasoning capability. To differentiate from inferable concepts, user-assisted concepts can be inferred to a certain extent, but not 100% inferred.

For example, although it might be tempting to computationally infer bathrooms from their size (i.e., bathrooms tend to be smaller than other rooms), this cannot guarantee a consistent result because storage rooms can also be small, whereas public restaurants can

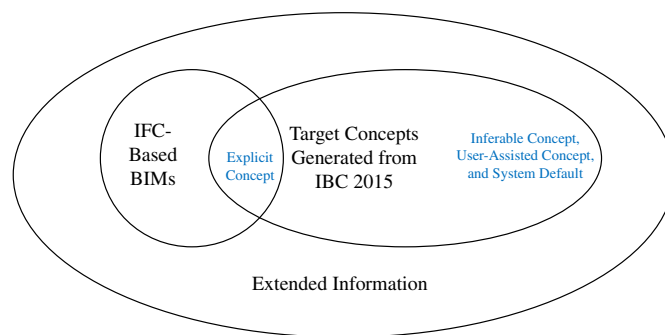


Fig. 3. Venn diagram of concepts in building codes and IFC models.

have quite large bathrooms. As such, additional manual input is needed to decide if a room is a bathroom. As a result, it is hard to quantify the size of the bathrooms, and relative size does not work either. Furthermore, room functions may change during the planning phase, so additional inputs from users are required for these concepts' identification. By allowing a certain extent of user judgment, the proposed method also incorporates some flexibility and error-tolerance in the building design. An important consideration is that the additional user input shall be minimal with inferred information computed to the largest extent possible, and the input should be collected preferably by multiple-choice or yes/no type of questions, with the least amount of domain knowledge and manual efforts needed.

The system defaults are the concepts that are not representing actual objects from a building, such as tables in the regulation, equations for calculations, references to other sections, or other concepts that are not directly related to BIMs, which, as a result, will not be the focus of this research. However, they are still needed in order for the downstream logic-based reasoning to execute successfully.

To illustrate the idea, Fig. 3 shows the relations between the four categories of target concepts based on the aforementioned IFC models and the ICC (2015). It also reveals that direct mapping from building codes to BIMs is not feasible for all concepts, and IFC models also contain information not used in the building code compliance checking. With the extended information, such gaps in matching BIMs with building code concepts can be addressed.

Step 3: Develop Algorithms for Extending Model Information to Match Target Concepts

In this step, information extension and matching algorithms are developed to match the invariant signatures generated from Step 1 to the target concepts identified in Step 2. Heuristic rule-based algorithms are developed iteratively until consistent results are obtained in the training data, i.e., the target concepts matching need to achieve 100% precision and recall in the training data.

For explicit concepts, the algorithms are straightforward in that each object will generate one instance of the corresponding target concept. For example, a door object (*IfcDoor*) from the IFC model can be used to directly generate an instance of the door concept. The attributes of the instance should also be extracted, such as the length, width, and others. These attribute values can be directly obtained from the invariant signatures of the object.

For inferable concepts, heuristic rule-based algorithms are developed to extend the explicit information to the target information by inference. Each concept has its own identification subalgorithm. For example, the egress concept (i.e., target concept) does not have a one-to-one mapping to existing IFC entities. The identification of

egresses therefore needs to be conducted through inference. To develop the algorithm, heuristic rules are developed first. For example, one possible heuristic rule for egress is that doors at boundaries of the building that connect interior and exterior of the building can be recognized as egresses. An algorithm can therefore be developed to identify a door's shape, position, level, and relative locations to decide if the door can be classified as an egress, using this heuristic rule. Then all inferred results are added to the extended information set (i.e., the information originated from the BIMs), so the extended information set now contains all the inferred information in addition to the original explicit information. All information is stored for later use.

For user-assisted concepts, a light user interface (UI) is developed to allow users to input the missing information. Inference algorithms are still developed to infer intermediate results, so that the questions presented to the user only ask for minimal input, and preferably using multiple-choice questions and/or binary yes/no questions. In addition to soliciting user inputs, the algorithms also combine the user input with existing explicit and inferred information to enrich the extended information set.

At this step, all system default concepts are identified to support further development. No further processing is needed from the model validation perspective because the information does not directly map to the BIMs per se. For example, Section_1003_3 is such a concept, which refers to Chapter 10, Section 1003.3 of a selected building code. Although such concepts do not have a direct mapping in the extended information, they are needed in supporting the later automated reasoning stage of code compliance checking.

Step 4: Generate Logic Facts to Store Extended Information Set

With the extended information set produced from the previous step, logic facts are generated to store this information. Instead of higher-order logic (Gallin 2011) and defeasible logic (Naeem 2014), the logic facts store all the instances of target concepts in FOL format, which is widely used and successfully tested for storing objects and relations to conduct deductive reasoning effectively and expressively (Zhang and El-Gohary 2015). It is machine-readable so that they can be directly used for logic-based automated code compliance reasoning. The advantage of this logic fact format is that it is very easy to read, edit, store, and manage. The logic facts can be directly fed into a logic-based building code compliance checking

system for automated reasoning, with no further processing needed on the model information side.

The logic facts follow a uniform representation of concepts and properties. For each instance of the target concept, a numbered instance is declared [e.g., *egress(egress32)*]. Then properties of the concept are linked to the instance in the form of logic relations [e.g., *has(egress32, height117)* indicates the height property of egress instance number 32 is represented by height instance number 117]. Different concepts may have their unique properties, e.g., the room type property applies to a room but does not apply to a window. For algorithm development of generating logic facts, each target concept uses its own designated subalgorithm. All generated logic facts are then manually checked for evaluation.

Experiment

For experimental testing, the authors collected five real commercial projects and a residential project, namely a convenience store, a warehouse, two restaurants (a fast-food restaurant and an Italian restaurant), a hotel, and a duplex apartment. The first five models were provided by our industry partner and all of them are located in Texas, and the last one is a sample residential building model from the open-source common BIM files site (WBDG 2021). All the six models, however, were designed by different architects. For each project, the authors used a corresponding building model in IFC format. The authors followed a 5:5 ratio for splitting the projects into training and testing models. As a result, the convenience store, warehouse, and the Italian restaurant were used as training data, whereas the fast-food restaurant, the hotel, and the duplex apartment models were used as testing data. Fig. 4 shows the training projects, and Fig. 5 shows the testing projects.

The hotel project contains a 4-story main building and two 1-story side buildings, which has the largest number of building elements and was also the largest in footage. The duplex apartment project contains a 2-story building. The remaining four commercial buildings are all 1-story buildings without side buildings. All models are well-functioned complete building models with multiple rooms, walls, doors, and others. For example, the Italian restaurant, convenience store, warehouse, fast-food restaurant, hotel, and duplex contained 14, 12, 1, 1, 200, and 11 rooms, 34, 37, 18, 54, 673, and 57 walls, and 13, 11, 34, 11, 317, and 14 doors, respectively. In summary, two of the testing models are similar to the three training

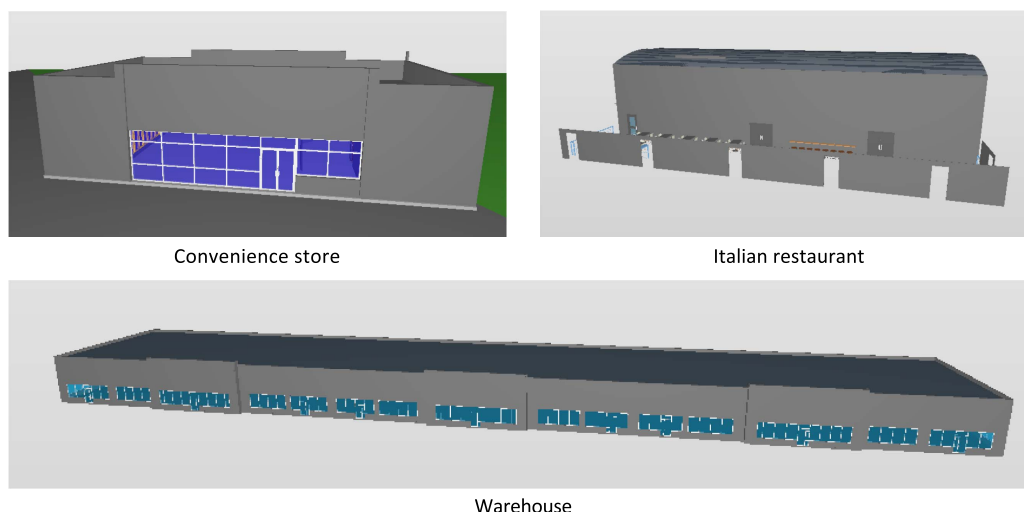


Fig. 4. Visualizations of training project models: convenience store, Italian restaurant, and warehouse.

models, which is 1- or 2-story but with different layouts and different numbers of rooms, whereas the other testing model (hotel project) is less similar in that it contains four stories and has significantly more rooms, which adds more complications in the validation process.

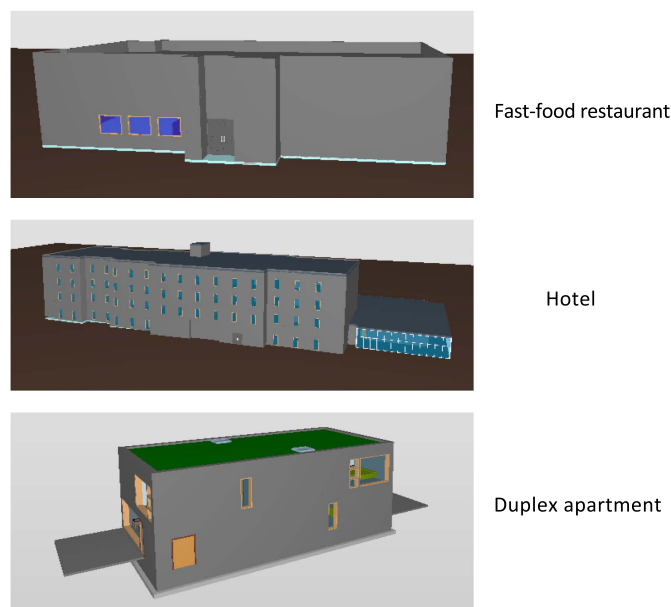


Fig. 5. Visualizations of testing project models: fast-food restaurant, hotel, and duplex apartment.

Table 1. Properties of invariant signatures-based features

Signature type	Example feature	Feature count
Geometric	Length, radius	21
Locational	Position, orientation direction	7
Metadata	Number of faces, average number of vertices of faces	8

Table 2. Count of concepts for each type

Concept type	Example concept	Concept count
Explicit	Wall, fence, window	8
Inferable	Egress, story	11
User-assisted	Bathroom, Occupied_roof	7
System default	Equation_20, message, Group_H	1,382

Table 3. All four types of concepts

Explicit concept	Inferable concept	User-assisted concept	System default
Floor	Building	Stairway_doors	Group_H
Wall	Building_height	Machinery_rooms	Means
Window	Floor_level	Storage_rooms	Barrier
Door	Floor_surface	Lobbies	Message
Room	Egress	Bathroom	1,378 more concepts
Ceiling	Assembly_areas	Exit_discharge_doors	—
Fence	Means_of_egress_system	Occupied_roof	—
Stair	Exit	—	—
—	Mezzanine	—	—
—	Room_elevations	—	—
—	Story	—	—

Step 1: Construct Invariant Signatures from IFC Models

For the model validation process, the first step is to preprocess the models to extract invariant signatures-based information from the IFC models.

In constructing the invariant signatures, the state-of-the-art algorithms (Wu et al. 2021) were used to extract values from the IFC models. For each AEC object, 36 features were generated as invariant signatures, including geometric, locational, and metadata signatures. The distribution of the invariant signatures-based features is presented in Table 1.

The invariant signatures were able to fully represent the information about building objects of a model. This simplified the process of further conversion because the invariant signatures were ready to be used in developing the heuristic rule-based classification algorithms. Later algorithms' development follows an iterative approach, i.e., for any information found missing in the later development phase, a refinement/extension of the invariant signatures was conducted.

Step 2: Identify and Classify Target Concepts

With the invariant signatures-based processing of IFC models completed, the building codes were then processed using the state-of-the-art information extraction and transformation algorithm (Zhang and El-Gohary 2015, 2016b) to generate logic rules from International Building Code 2015 (ICC 2015). To illustrate the idea, the authors chose Chapter 10 for demonstration. The automatically generated logic rules were then manually refined to error-freely represent the regulatory requirements in Chapter 10 of IBC 2015.

To extract the target concepts from the logic rules, the authors developed a target concept identifier that can identify a target concept by recognizing the single-variable conjunct pattern of a logic clause [i.e., conjunct of the form *entity(Entity)*]. For example, a *door(Door)* conjunct indicates a declaration of a door object variable. For each target concept, the logic rules always contain this type of declaration. Therefore, the target concept identification algorithm could identify all the related building code concepts from the logic rules.

With this target concept identification algorithm, the authors were able to identify 1,408 target concepts. These concepts were then classified into the four types, as indicated in Tables 2 and 3. For the concept coverage of models, every model covered most of the 1,408 concepts from Chapter 10 of IBC 2015. The exceptions were fence, stair, and mezzanine. Only the Italian restaurant contained fences. Only the hotel model and the duplex apartment model contained stairs. No model contained mezzanines.

Table 4. Explicit concepts identification result on training models

Concept	IFC map	No. identified	No. correctly identified	Gold standard	Precision (%)	Recall (%)
Floor	IfcSlab	24	24	24	100	100
Wall	IfcWall/IfcWallStandardCase	99	99	99	100	100
Window	IfcWindow	20	20	20	100	100
Door	IfcDoor	58	58	58	100	100
Room	IfcSpace	27	27	27	100	100
Ceiling	IfcRoof	5	5	5	100	100
Fence	IfcRailing	12	12	12	100	100
Stair	IfcStair	0	0	0	—	—
Total	—	245	245	245	100	100

Step 3: Develop Algorithms for Extending Model Information to Match Target Concepts

Explicit Concepts

Explicit concepts were straightforward in their identification algorithm development, as the name suggested. Each explicit concept had a one-to-one mapping to the IFC entities, which was directly used for identifying them in the IFC models. For example, doors were identified through *IfcDoor*, and rooms were identified through *IfcSpace*.

The validation of these concepts' identification was conducted by directly checking the number and properties of the corresponding IFC entities. Results showed all the needed explicit concepts were identified with 100% precision and 100% recall in the training data (Table 4).

Inferable Concepts

For inferable concepts, one of the most representative concepts is egress. The authors conducted an experiment following the heuristic described in the Proposed Method section. The detailed implementation was as follows:

Step 1: Identify the boundary of the building. The boundary is found by taking the maximum and minimum coordinates of walls and slabs on the first floor of the building. Four line segments need to be identified for each building model.

Step 2: Select doors that are at the boundary of the building. This is achieved by checking the position of the doors. The doors selected are the ones that are close (i.e., within a predefined threshold) to the boundary of the building.

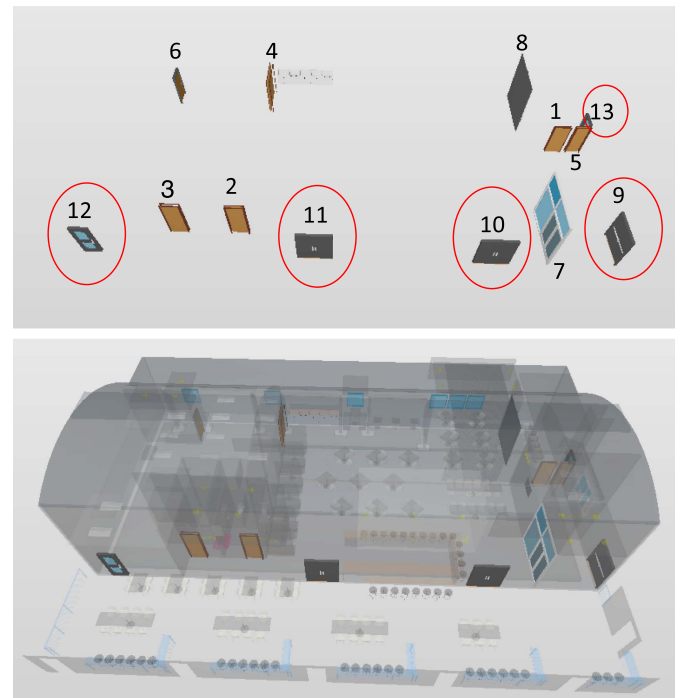
Step 3: Verify the orientation of each door selected from Step 2 and keep the ones that connect the interior and exterior of a building. This is achieved by checking if the door's orientation is in parallel to the corresponding boundary.

The visualization of the egress identification results on the Italian restaurant model is shown in Fig. 6.

As illustrated in Fig. 6, Doors 1, 2, 3, 4, 6, and 8 were not identified as egress because they were not at the boundary of the building (Step 2 criterion was not satisfied). Doors 5 and 7 were further eliminated because they failed to satisfy the orientation criterion at Step 3, i.e., the direction was not in parallel with the corresponding boundary. Finally, Doors 9, 10, 11, 12, and 13 were identified as egresses because they met all the three criteria in the algorithm.

In addition to egress, the authors also developed algorithms for the rest of the inferable concepts (Table 5).

For example, for the identification of a building, the main heuristic used was to identify the boundary of the building based on a range for all involved building objects. For identifying floor levels, clustering algorithm was used to cluster slab objects together based on their elevation. For identifying a mezzanine, the main heuristic used was to recognize the slab objects between floor levels.

**Fig. 6.** Visualization of all the egresses identified for Italian restaurant.**Table 5.** Heuristics used in inferable concepts identification algorithms

Concept	Heuristics description
Building	Identify the boundary
Building_height	One of the properties of the building concept
Floor_level	Using clustering algorithm to identify levels
Floor_surface	One of the properties of the floor concept
Egress	Door entity at the boundary that connects interior and exterior
Assembly_areas	One of the properties of the building concept
Means_of_egress_system	Same as egress
Exit	Same as egress
Mezzanine	Floor objects between floor levels
Room_elevations	One of the properties of the room concept
Story	Same as floor level

The results of the algorithm development for inferable concepts are presented in Table 6.

User-Assisted Concepts

For the other subtype of nonexplicit concepts, namely user-assisted concepts, the information from the IFC models was not sufficient

Table 6. Inferable concepts identification results on training models

Concept	No. identified	No. correctly identified	Gold standard	Precision (%)	Recall (%)
Building	3	3	3	100	100
Building_height	3	3	3	100	100
Floor_level	3	3	3	100	100
Floor_surface	3	3	3	100	100
Egress	42	42	42	100	100
Assembly_areas	3	3	3	100	100
Means_of_egress_system	42	42	42	100	100
Exit	42	42	42	100	100
Mezzine	0	0	0	—	—
Room_elevations	29	29	29	100	100
Story	3	3	3	100	100
Total	173	173	173	100	100

Table 7. Additional information needed for user-assisted concepts

Concept	Inferred concept	Additional information	Input type
Machinery_rooms	Room	Room type	Binary (yes/no)
Storage_rooms	Room	Room type	Binary
Lobbies	Room	Room type	Binary
Bathroom	Room	Room type	Binary
Exit_discharge_doors	Egress	Egress type	Binary
Occupied_roof	Roof	If the roof is occupied	Binary
Stairway_doors	Door	Door types	Binary

Table 8. User-assisted concepts identification result on training models

Concept	No. questions	No. correct questions	Gold standard	Precision (%)	Recall (%)
Machinery_rooms	27	27	27	100	100
Storage_rooms	27	27	27	100	100
Lobbies	27	27	27	100	100
Bathroom	27	27	27	100	100
Exit_discharge_doors	12	12	12	100	100
Occupied_roof	5	5	5	100	100
Stairway_doors	58	58	58	100	100
Total	183	183	183	100	100

to automatically identify those concepts even with inference. However, the inferences were able to be conducted to an extent to narrow down the range of selection. For example, based on the observation of the models, there was not enough information for differentiating bathrooms from other rooms. For the target concept of a bathroom, however, it can be inferred that it must be a room, which could help limit the range. Therefore, to identify bathrooms, the algorithm will display questions to the user to help filter through the narrowed range, e.g., “Is room2 a bathroom? Enter 1 for Yes, enter 2 for No.” Similarly, algorithms for processing other user-assisted concepts were developed with interactive questions.

Table 9. Number of logic clauses in the generated logic facts on training models

Concept type	No. logic clauses	No. correct logic clauses	Gold standard	Precision (%)	Recall (%)
Explicit	3,458	3,458	3,458	100	100
Inferable	699	699	699	100	100
User-assisted	80	80	80	100	100
System defaults	4,146	4,146	4,146	100	100
Total	8,333	8,333	8,333	100	100

Table 7 gives the additional information needed from users for successful identification of the user-assisted concepts. Table 8 provides the results of identifying user-assisted concepts.

System Defaults

As described previously, system defaults were not representing actual objects from a building design and therefore do not have (or need to have) any mapping in the extended information set. As a result, during the model validation process, these system defaults were identified, but no further action was conducted on them. They can be used to support the later compliance checking stage by instantiating logic rules for automated reasoning execution. For example, an equation concept referred to an equation from the building codes. A fully automated compliance checking system would need to use the equation for calculations in the checking, the process of which should also be automated. In line with such an approach, all 1,382 system defaults concepts were identified for future use.

Step 4: Generate Logic Facts to Store Extended Information Set

With all the algorithms for extending the information set, identified target concepts were stored in logic facts. The logic facts can be directly used for later logic-based automated code compliance reasoning.

To generate logic facts, each instance of the target concept from the extended information set was generated into connected B-Prolog (a Prolog system implementation with extensions for programming concurrency, constraints, and interactive graphics) clauses (Zhou 2014), including its declaration and properties, where Prolog is a FOL-based logic programming implementation. For example, the identified egress entity *1qPjXNL6r8rRF28rPA_nZx* (Door 10 in Fig. 6) was stored in “*egress(egress2). height(height32). has(egress2, height32). has_value(height32, 2.32). has_unit(height32, meter). ...*” Here, *egress2* was used because it was the second identified egress among all the identified egresses. The height of the egress was 2.23 m. To represent such information, a height instance was declared as *height32* because there were already 31 height instances declared beforehand. The *has_value(height32, 2.23).* and *has_unit(height32, meter).* define the value and unit of the height property, respectively. Additional information was also represented in the same format, such as its length, width, position, orientation direction, and others. Table 9 gives the results of the logic clauses generated from the extended information set.

Results and Analysis

To evaluate the robustness of the proposed method and developed algorithms, the authors tested them on the three testing models, namely the fast-food restaurant model, hotel model, and duplex apartment model (Fig. 5). The results are presented in Tables 10–13. Table 10 displays the results on explicit concepts. The hotel model

Table 10. Explicit concepts identification results on testing models

Concept	IFC map	No. identified	No. correctly identified	Gold standard	Precision (%)	Recall (%)
Floor	IfcSlab	39	39	39	100	100
Wall	IfcWall/IfcWallStandardCase	795	795	795	100	100
Window	IfcWindow	145	145	145	100	100
Door	IfcDoor	344	344	344	100	100
Room	IfcSpace	212	212	212	100	100
Ceiling	IfcRoof	6	6	6	100	100
Fence	IfcRailing	0	0	0	—	—
Stair	IfcStair	8	8	8	100	100
Total	—	1,549	1,549	1,549	100	100

Table 11. Inferable concepts identification result on testing models

Concept	No. identified	No. correctly identified	Gold standard	Precision (%)	Recall (%)
Building	3	3	5	100	60.0
Building_height	3	3	5	100	60.0
Floor_level	7	7	7	100	100
Floor_surface	39	39	39	100	100
Egress	16	12	17	75	70.6
Assembly_areas	3	2	3	66.7	66.7
Means_of_egress_system	16	12	17	75	70.6
Exit	16	12	17	75	70.6
Mezzanine	0	0	0	—	—
Room_elevations	212	212	212	100	100
Story	7	7	7	100	100
Total	322	309	329	96.0	93.9

Table 12. User-assisted concepts identification results on testing models

Concept	No. questions	No. correct questions	Gold standard	Precision (%)	Recall (%)
Machinery_rooms	212	212	212	100	100
Storage_rooms	212	212	212	100	100
Lobbies	212	212	212	100	100
Bathroom	212	212	212	100	100
Exit_discharge_doors	344	344	344	100	100
Occupied_roof	6	6	6	100	100
Stairway_doors	344	344	344	100	100
Total	1,542	1,542	1,542	100	100

had four roofs that were not directly observable in Fig. 5, so the authors showed the roofs again in Fig. 7.

Table 11 provides the results on inferable concepts. The precision was 96.0% and the recall was 93.9%. As an example illustration, the egresses identified are shown in Figs. 8 and 9 for the fast-food restaurant model and hotel model, respectively.

For fast-food restaurant model, both egresses were successfully identified. For the hotel model (Fig. 9), six egresses were successfully identified, whereas five egresses were missed. This error

occurred because there were three separate buildings in the hotel model, whereas the algorithm only identified one building, the main building. A further analysis showed that this was because in all three training models, there was only one building for each model. Following the data-driven approach, the assumption of one building for each model was established from the training models, whereas it does not hold in the hotel testing model. As a result, the trained algorithms in our experiment failed to identify the other two buildings and the corresponding five egresses. In addition, one assembly area was also incorrectly identified because of this error.

Table 12 presents the results of user-assisted concepts. The developed algorithms produced simple binary (yes/no) questions based upon which perfect precision and recall could be achieved. Table 13 gives the generated logic facts for instances of all four types of concepts in the testing models.

As indicated in Table 13, the algorithms achieved an overall 99.8% precision and 99.6% recall, which is very promising. However, for the inferable concept category, the precision was only 96.8%, and the recall was only 93.9% in the generated logic facts. The logic clause-level recall was different from the concept-level recall in Table 11 because one egress concept needs to be represented by multiple logic clauses.

The error in the inferable concept category occurred because the developed system did not identify all three buildings in the hotel model, which can be seen in Figs. 7 and 9. A further inspection showed that the developed algorithm used a relatively strong assumption that a building is close to a rectangular shape (with some allowable error margin) and there is only one building in the model. This assumption was valid during the training phase because all training models were single-building models with a rectangular-shaped boundary. However, this assumption was not valid on the testing hotel model because there were two side buildings attached to the main building. Therefore this hotel model did not have a one-to-one mapping between buildings and *IfcBuilding* entities (i.e., there was only one *IfcBuilding* entity but three connected buildings). This unseen multibuilding model resulted in an error in the developed system and caused a 3.2% drop in precision and a 6.1% drop in recall on the logic clauses of the inferable concept category.

To resolve it, the authors did a follow-up experiment to modify the algorithm to allow a polyline shape as building boundaries and

Table 13. Number of logic clauses in the generated logic facts on testing models

Concept type	No. logic clauses	No. correct logic clauses	Gold standard	Precision (%)	Recall (%)
Explicit	22,316	22,316	22,316	100	100
Inferable	1,617	1,565	1,667	96.8	93.9
User-assisted	84	84	84	100	100
System defaults	4,146	4,146	4,146	100	100
Total	28,163	28,111	28,213	99.8	99.6

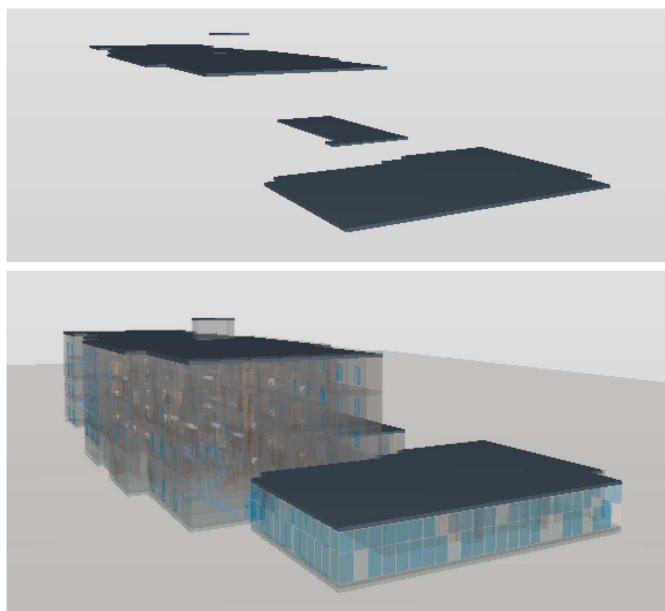


Fig. 7. Visualization of the four roofs identified for the hotel model.

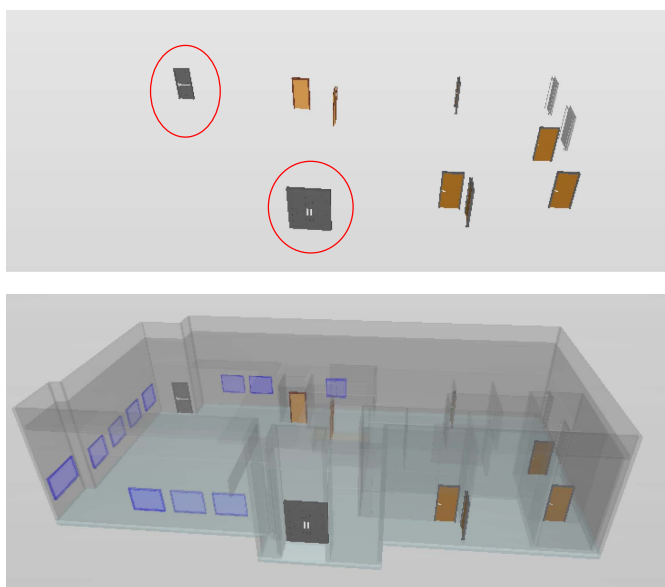


Fig. 8. Visualization of the two egresses identified for the fast-food restaurant model.

allow multiple buildings to exist in one project model. This modification enabled the algorithm to then correctly identify all buildings and egresses. Based on the nature of rule-based algorithms, the same level of performance may not be readily achieved on unseen patterns. However, with further development on more training data to cover more concepts and patterns for building code and building design models, respectively, the expectation is that the accumulated set of algorithms will continuously be expanded and therefore enhance its robustness to asymptotically approach the ultimate super-set of algorithms.

In Chapter 10 of ICC (2015), 1,408 regulatory concepts can be extracted, which is sufficient to demonstrate the robustness of the method. This method is expected to generate comparable

results on other chapters of the IBC 2015. For other chapters in the IBC 2015, there are many concepts that were in common with Chapter 10, such as building, wall, ceiling, and others. For new concepts, the proposed method can also be used to categorize them and develop new identification algorithms accordingly.

In summary, the proposed method achieved an overall 99.8% precision and 99.6% recall in the resulting logic clauses.

Contributions to the Body of Knowledge

The contributions to the body of knowledge are fourfold. First, the authors investigated the feasibility of automated inference of non-explicit information in BIMs to match with building code concepts. It was found that this feasibility depends on the type of nonexplicit information to be derived. For nonexplicit information that is not directly inferable, the authors still leveraged inference to reduce the needed manual input from users. Second, the authors presented a new categorization of the concepts from building codes into four categories of (1) explicit, (2) inferable, (3) user-assisted, and (4) system defaults, in the context of serving as target concepts of validating/matching with BIMs, to facilitate the analysis of BIMs for ACC in a divide-and-conquer manner.

Third, in the identification of concepts from BIMs, the authors developed invariant signatures–based algorithms for matching with each of the four categories of building code concepts, with high recall and precision. Last but not least, the proposed method was able to extend the information from BIMs and check for missing information in the context of ACC. Combined with logic rule-based algorithms, the resulting information set (i.e., collected information) was demonstrated to be significantly extended to facilitate the matching of BIMs with building codes. In summary, the authors made a solid advancement in filling the gap of information mapping from BIMs to building codes for ACC, by introducing an extended information set using invariant signatures–based inferences. This method supports model validation in ACC and therefore indirectly and positively affects interoperability of BIM.

Conclusions

The authors proposed an iterative method for BIMs model validation using invariant signatures and logic inference to support automated building code compliance checking. The proposed method can extend the information extracted from BIMs with nonexplicit concepts to support mapping from BIMs to target concepts in building codes. The proposed method uses invariant signatures of AEC objects as intermediate results to extract information from IFC entities and extended the extracted information by inferring more concepts based on heuristic rule-based algorithms and user inputs. An experiment on IBC 2015 and five commercial building models showed that the proposed method achieved 99.8% precision and 99.6% recall in generating the extended information set. The proposed method was shown to function with consistent results between testing data and training data, in mapping the extended information set from BIMs to building code concepts. This helps address the existing research gap of BIM validation to support fully automated building code compliance checking.

Limitations and Future Work

The authors acknowledge the following limitations of this paper:

- The proposed method was tested in only one chapter (i.e., Chapter 10) of IBC 2015; how it will perform in other

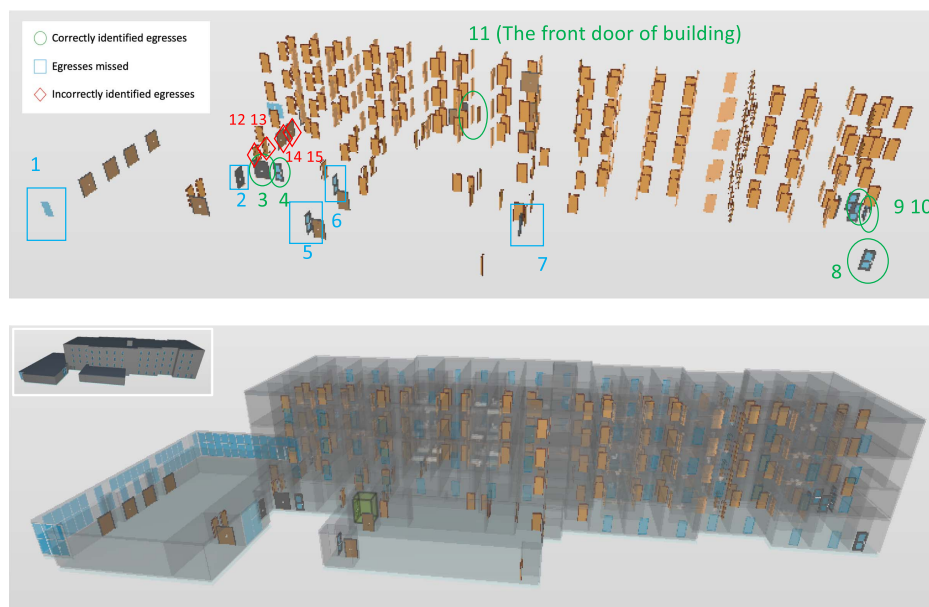


Fig. 9. Visualization of all egresses identified for the hotel model (viewed from the backside of the building). Egresses 3, 4, 8, 9, 10, and 11 were correctly identified, whereas Egresses 1, 2, 5, 6, and 7 were missed, and Egresses 12, 13, 14, and 15 were incorrectly identified.

chapters and other building codes remain to be tested. However, the authors would expect comparable results on other chapters and other building codes using the same method.

- In order for the processing of inferable concepts to be error-free, the assumption used in the heuristic rules needs to be broadly applicable in any foreseeable BIMs.
- Due to the nature of rule-based algorithms, it is time-consuming and labor-intensive to grow the number of concepts and patterns covered, whereas error is always possible before the number of concepts and patterns saturate. However, the authors believe the number of concepts and patterns in this context is enumerable, and a compatible set of rules are feasible with careful implementation, such as by performing a comprehensive check after each new rule is added. In addition, although the theoretical optimal rule set may be hard to achieve, it is always feasible to arrive at practically comprehensive solutions for a known scope [e.g., concepts in the ICC (2018)] using data-driven approaches.
- The method was validated only on commercial and residential buildings. Other types of built environment, such as civil infrastructure (e.g., bridges, railway stations) remain to be tested. In their future research, the authors plan to test their method on more building codes and explore ways to ensure the broad applicability and compatibility of heuristic rules. Furthermore, more models will be used to accumulatively and continuously expand the set of algorithms and building code concepts covered.

Data Availability Statement

All data that support the findings of this study are available from the corresponding author upon reasonable request.

Acknowledgments

The authors would like to thank the National Science Foundation (NSF). This material is based on work supported by the NSF under

Grant No. 1827733. Any opinions, findings, and conclusions or recommendations expressed in this material are those of the authors and do not necessarily reflect the views of the NSF.

References

- Adán, A., B. Quintana, S. A. Prieto, and F. Bosché. 2018. "Scan-to-BIM for 'secondary' building components." *Adv. Eng. Inf.* 37: 119–138. <https://doi.org/10.1016/j.aei.2018.05.001>.
- Azhar, S. 2011. "Building information modeling (BIM): Trends, benefits, risks, and challenges for the AEC industry." *Leadersh. Manage. Eng.* 11 (3): 241–252. [https://doi.org/10.1061/\(ASCE\)LM.1943-5630.0000127](https://doi.org/10.1061/(ASCE)LM.1943-5630.0000127).
- Ding, L., R. Drogemuller, M. Rosenman, D. Marchant, and J. Gero. 2006. "Automating code checking for building designs: Design check." In *Clients driving innovation: Moving ideas into practice*, 1–16. Brisbane, Australia: Construction Innovation.
- Ding, L., K. Li, Y. Zhou, and P. E. D. Love. 2017. "An IFC-inspection process model for infrastructure projects: Enabling real-time quality monitoring and control." *Autom. Constr.* 84 (Dec): 96–110. <https://doi.org/10.1016/j.autcon.2017.08.029>.
- Eastman, C., J. Lee, Y. Jeong, and J. Lee. 2009. "Automatic rule-based checking of building designs." *Autom. Constr.* 18 (8): 1011–1033. <https://doi.org/10.1016/j.autcon.2009.07.002>.
- Fenves, S. J. 1966. "Tabular decision logic for structural design." *J. Struct. Div.* 92 (6): 473–490. <https://doi.org/10.1061/JSDEAG.0001567>.
- Fernández, M., I. Cantador, V. López, D. Vallet, P. Castells, and E. Motta. 2011. "Semantically enhanced information retrieval: An ontology-based approach." *Web Semant.* 9 (4): 434–452. <https://doi.org/10.1016/j.websem.2010.11.003>.
- Gallin, D. 2011. *Intensional and higher-order modal logic with applications to Montague semantics*. Amsterdam, Netherlands: Elsevier Science.
- Garrett, J. H., Jr., and S. J. Fenves. 1987. "A knowledge-based standard processor for structural component design." *Eng. Comput.* 2 (4): 219–238. <https://doi.org/10.1007/BF01276414>.
- Hamledari, H., B. McCabe, and S. Davari. 2017. "Automated computer vision-based detection of components of under-construction indoor partitions." *Autom. Constr.* 74 (Feb): 78–94. <https://doi.org/10.1016/j.autcon.2016.11.009>.

- Haubler, M., S. Esser, and A. Borrmann. 2021. "Code compliance checking of railway designs by integrating BIM, BPMN and DMN." *Autom. Constr.* 121 (Jan): 103427. <https://doi.org/10.1016/j.autcon.2020.103427>.
- Hjelseth, E., and N. Nisbet. 2011. "Capturing normative constraints by use of the semantic mark-up (RASE) methodology." In *Proc., CIB W78 2011—28th Int. Conf.-Applications of IT in the AEC Industry*. Rotterdam, Netherlands: Conseil International du Bâtiment.
- Holzer, D. 2015. *BIM manager's handbook. Best practice BIM. EPart 1*. West Sussex, UK: Wiley.
- ICC (International Code Council). 2009. "2009 International Building Code." Accessed November 4, 2021. <https://codes.iccsafe.org/content/IBC2009>.
- ICC (International Code Council). 2015. "2015 International Building Code." Accessed November 4, 2021. <https://codes.iccsafe.org/content/IBC2015>.
- ICC (International Code Council). 2018. "2018 International Fire Code." Accessed November 4, 2021. <https://codes.iccsafe.org/content/IFC2018>.
- Jeong, J., and G. Lee. 2009. "Requirements for automated code checking for fire resistance and egress rule using BIM." In *Proc., Int. Conf. on Construction Engineering and Project Management*, 316–322. Seoul, Korea: Korea Construction Management Association.
- Kasim, T., H. Li, Y. Rezgui, and T. Beach. 2013. "Automated sustainability compliance checking process: Proof of concept." Accessed January 2, 2021. <http://itc.scix.net/paper/convr-2013-1>.
- Khemlani, L. 2011. "CORENET e-PlanCheck: Singapore's automated code checking system. AECbytes." Accessed January 2, 2021. <http://www.aecbytes.com/feature/2005/CORENETePlanCheck.html>.
- Kim, H., Z. Shen, I. Kim, K. Kim, A. Stumpf, and J. Yu. 2016. "BIM IFC information mapping to building energy analysis (BEA) model with manually extended material information." *Autom. Constr.* 68 (Aug): 183–193. <https://doi.org/10.1016/j.autcon.2016.04.002>.
- Lopez, L., S. Elam, and K. Reed. 1989. "Software concept for checking engineering designs for conformance with codes and standards." *Eng. Comput.* 5 (2): 63–78. <https://doi.org/10.1007/BF01199070>.
- Lopez, L. A., and R. N. Wright. 1985. *Mapping principles for the standards interface for computer aided design*. Gaithersburg, MD: National Bureau of Standards.
- Martins, J. P., and A. Monteiro. 2013. "LicA: A BIM based automated code-checking application for water distribution systems." *Autom. Constr.* 29 (Jan): 12–23. <https://doi.org/10.1016/j.autcon.2012.08.008>.
- Nadkarni, P. M., L. Ohno-Machado, and W. W. Chapman. 2011. "Natural language processing: An introduction." *J. Am. Med. Inf. Assoc.* 18 (5): 544–551. <https://doi.org/10.1136/amiajnl-2011-000464>.
- Naeem, K. J. 2014. *A defeasible logic programming-based framework to support argumentation in semantic web applications*. New York: Springer.
- Nawari, O. 2012. "Automating codes conformance." *J. Archit. Eng.* 18 (4): 315–323. [https://doi.org/10.1061/\(ASCE\)AE.1943-5568.0000049](https://doi.org/10.1061/(ASCE)AE.1943-5568.0000049).
- Nawari, O. 2018. *Building information modeling: Automated code checking and compliance processes*. Boca Raton, FL: CRC Press.
- Nguyen, T., and J. Kim. 2011. "Building code compliance checking using BIM technology." In *Proc., 2011 Winter Simulation Conf. (WSC)*, 3395–3400. New York: IEEE.
- Paliouras, G., C. D. Spyropoulos, and G. Tsatsaronis. 2011. *Knowledge-driven multimedia information extraction and ontology evolution: Bridging the semantic gap*. Berlin: Springer.
- Puente, I., H. González-Jorge, J. Martínez-Sánchez, and P. Arias. 2014. "Automatic detection of road tunnel luminaires using a mobile LiDAR system." *Meas. J. Int. Meas. Confederation* 47 (1): 569–575. <https://doi.org/10.1016/j.measurement.2013.09.044>.
- Quintana, B., S. A. Prieto, A. Adán, and F. Bosché. 2018. "Door detection in 3D coloured point clouds of indoor environments." *Autom. Constr.* 85 (Jan): 146–166. <https://doi.org/10.1016/j.autcon.2017.10.016>.
- Sacks, R. 2018. *BIM handbook: A guide to building information modeling for owners, designers, engineers, contractors and facility managers*. Hoboken, NJ: Wiley.
- Sarawagi, S. 2008. "Information extraction." *Found. Trends Databases* 1 (3): 261–377. <https://doi.org/10.1561/19000000003>.
- Sionov, R. V., S. A. Vlahopoulos, and Z. Granot. 2015. "Regulation of BIM in health and disease." *Oncotarget* 6 (27): 23058–23134. <https://doi.org/10.18632/oncotarget.5492>.
- Solihin, W., and C. Eastman. 2015. "Classification of rules for automated BIM rule checking development." *Autom. Constr.* 53 (May): 69–82. <https://doi.org/10.1016/j.autcon.2015.03.003>.
- Tan, X., A. Hammad, and P. Fazio. 2010. "Automated code compliance checking for building envelope design." *J. Comput. Civ. Eng.* 24 (2): 203–211. [https://doi.org/10.1061/\(ASCE\)0887-3801\(2010\)24:2\(203\)](https://doi.org/10.1061/(ASCE)0887-3801(2010)24:2(203)).
- Volk, R., J. Stengel, and F. Schultmann. 2014. "Building information modeling (BIM) for existing buildings—Literature review and future needs." *Autom. Constr.* 38 (Mar): 109–127. <https://doi.org/10.1016/j.autcon.2013.10.023>.
- WBDG (Whole Building Design Guide). 2021. "Common building information model files and tools." Accessed July 20, 2021. <https://www.wbdg.org/bim/cobie/common-bim-files>.
- Wimalasuriya, D. C., and D. Dou. 2010. "Ontology-based information extraction: An introduction and a survey of current approaches." *J. Inf. Sci.* 36 (3): 306–323. <https://doi.org/10.1177/0165551509360123>.
- Wu, J., H. L. Sadraddin, R. Ren, J. Zhang, and X. Shao. 2021. "Invariant signatures of architecture, engineering, and construction objects to support BIM interoperability between architectural design and structural analysis." *J. Constr. Eng. Manage.* 147 (1): 04020148. [https://doi.org/10.1061/\(ASCE\)CO.1943-7862.0001943](https://doi.org/10.1061/(ASCE)CO.1943-7862.0001943).
- Wu, J., and J. Zhang. 2019a. "Introducing geometric signatures of architecture, engineering, and construction objects and a new BIM dataset." In *Proc., 2019 ASCE Int. Conf. on Computing in Civil Engineering*, 264–271. Reston, VA: ASCE.
- Wu, J., and J. Zhang. 2019b. "New automated BIM object classification method to support BIM interoperability." *J. Comput. Civ. Eng.* 33 (5): 04019033. [https://doi.org/10.1061/\(ASCE\)CP.1943-5487.0000858](https://doi.org/10.1061/(ASCE)CP.1943-5487.0000858).
- Xu, X., and H. Cai. 2020. "Semantic approach to compliance checking of underground utilities." *Autom. Constr.* 109 (Jan): 103006. <https://doi.org/10.1016/j.autcon.2019.103006>.
- Yang, Q. Z., and X. Xu. 2004. "Design knowledge modeling and software implementation for building code compliance checking." *Build. Environ.* 39 (6): 689–698. <https://doi.org/10.1016/j.buildenv.2003.12.004>.
- Yang, Z., H. Yu, J. Tang, and H. Liu. 2019. "Toward keyword extraction in constrained information retrieval in vehicle social network." *IEEE Trans. Veh. Technol.* 68 (5): 4285–4294. <https://doi.org/10.1109/TVT.2019.2906799>.
- Yehia, E., E. Boshnak, S. AbdelGaber, A. Abdo, and D. S. Elzanfaly. 2019. "Ontology-based clinical information extraction from physician's free-text notes." *J. Biomed. Inf.* 98 (Oct): 103276. <https://doi.org/10.1016/j.jbi.2019.103276>.
- Zhang, J., and N. El-Gohary. 2015. "Automated information transformation for automated regulatory compliance checking in construction." *J. Comput. Civ. Eng.* 29 (4): B4015001. [https://doi.org/10.1061/\(ASCE\)CP.1943-5487.0000427](https://doi.org/10.1061/(ASCE)CP.1943-5487.0000427).
- Zhang, J., and N. El-Gohary. 2016a. "Extending building information models semi-automatically using natural language processing techniques." *J. Comput. Civ. Eng.* 30 (5): C4016004. [https://doi.org/10.1061/\(ASCE\)CP.1943-5487.0000536](https://doi.org/10.1061/(ASCE)CP.1943-5487.0000536).
- Zhang, J., and N. El-Gohary. 2016b. "Semantic NLP-based information extraction from construction regulatory documents for automated compliance checking." *J. Comput. Civ. Eng.* 30 (2): 04015014. [https://doi.org/10.1061/\(ASCE\)CP.1943-5487.0000346](https://doi.org/10.1061/(ASCE)CP.1943-5487.0000346).
- Zhang, J., and N. El-Gohary. 2016c. "Semantic-based logic representation and reasoning for automated regulatory compliance checking." *J. Comput. Civ. Eng.* 31 (1): 4016037. [https://doi.org/10.1061/\(ASCE\)CP.1943-5487.0000583](https://doi.org/10.1061/(ASCE)CP.1943-5487.0000583).
- Zhang, J., and N. El-Gohary. 2017. "Integrating semantic NLP and logic reasoning into a unified system for fully-automated code checking."

Autom. Constr. 73 (Jan): 45–57. <https://doi.org/10.1016/j.autcon.2016.08.027>.

Zhang, R., and N. El-Gohary. 2019. “A machine-learning approach for semantic matching of building codes and building information models (BIMs) for supporting automated code checking.” In *Proc., GeoMEast 2019 Int. Congress and Exhibition*. New York: Springer.

Zhou, N. 2014. “B-Prolog user’s manual (version 8.1): Prolog, agent, and constraint programming.” Accessed April 1, 2021. <http://www.picat-lang.org/bprolog/download/manual.pdf>.

Zou, Y., A. Kiviniemi, and S. W. Jones. 2017. “A review of risk management through BIM and BIM-related technologies.” *Saf. Sci.* 97 (Aug): 88–98. <https://doi.org/10.1016/j.ssci.2015.12.027>.